

Material de Estudio Profundo — Deck EFT (12 láminas)

Objetivo: que al ver "lámina X" ya sepas la historia completa y hables sin mirar el contenido. Cada lámina tiene:  el concepto (la teoría),  la historia en nuestro sistema (cómo lo vivimos y aplicamos),  hilos para explainarte, y  el ancla — una frase que resume todo.

Lámina 01 — Portada

 **Ancla:** "Anticipamos qué clientes se van, antes de que se vayan."

 **El concepto.** El proyecto completo es un ciclo de vida del dato: dato crudo → pipeline → base de datos → modelo → predicciones → dashboard → decisión de negocio. La portada presenta ese sistema, no una tarea de curso. El stack (GitHub · Railway · Supabase · Streamlit) no es decorado: cada logo es una capa con una responsabilidad (código+CI, cómputo, datos, visualización).

 **La historia.** Partimos en la Evaluación 1 con un caso IoT y pivotamos a Telco Churn porque tenía un dataset real (7.043 clientes), una variable objetivo clara ([Churn](#)) y un problema de negocio medible. Desde la Ev2 todo se construyó con la misma filosofía: **desacoplado, reproducible y auditable**. Roles del equipo: Benjamín como Ingeniero de Datos, Diego como Ingeniero de BD/DataOps — pero ambos dominan todo (el profe puede preguntarle cualquier parte a cualquiera).

 **Para explainarte:** si te piden el elevator pitch: "Una telco pierde 1 de cada 4 clientes al año y captar uno nuevo cuesta 5-7 veces más que retener. Construimos un sistema que ordena la cartera por riesgo de fuga con 79,7% de recall, para que retención actúe antes y focalizado." Menciona metodología PMBOK híbrida si preguntan cómo se gestionó (ver §PMBOK al final).

Lámina 02 — Punto de partida (pipeline DataOps + vs tradicional)

 **Ancla:** "El modelo es tan bueno como el dato que lo alimenta — por eso la Fase 1 existe."

 **El concepto.** DataOps = aplicar ingeniería de software al ciclo del dato: **automatización** (nadie procesa a mano), **reproducibilidad** (la misma entrada produce siempre la misma salida), **testing** (validaciones automáticas), **versionado** (git) y **auditoría** (logs). El enfoque tradicional es el contraejemplo: un analista abre el CSV en Excel, limpia a criterio, nadie sabe qué cambió, y si se repite el proceso sale distinto. La diferencia no es de herramientas, es de **confiabilidad**: en tradicional los errores se descubren tarde (o nunca); en DataOps el pipeline los detecta y los deja registrados.

 **La historia en nuestro sistema.** Nuestras 4 etapas y qué hace cada una de verdad: 1. **Ingesta** ([src/ingesta.py](#)): trae el CSV fuente a la zona [raw](#) con timestamp. Cada corrida queda fechada — linaje desde el origen. 2. **Limpieza** ([src/limpieza.py](#)): tipos correctos (TotalCharges venía como texto con espacios), booleanos normalizados, [tenure_group](#) derivado. Salida a zona [clean](#). 3. **Validación** ([src/validaciones.py](#)): reglas estructurales (columnas, tipos) y semánticas (rangos, dominios). **Los que fallan no se botan:** van a [clientes_rechazados](#) con su motivo — auditoría de calidad. 4. **Carga** ([src/carga_bd.py](#)): **full-refresh**

idempotente y transaccional — TRUNCATE + INSERT en la misma transacción; si algo falla, ROLLBACK deja la tabla intacta. Cada corrida registra en `carga_logs`: leídos, insertados, rechazados, duración, estado.

Las mejoras del docente que incorporamos (esto suma en #4 y #7): tras la Parcial 2 el feedback fue "esto es un monolito" → lo desacoplamos: cada etapa corre como **un contenedor por capa** (docker-compose), la API en Railway, los datos en Supabase, el código en GitHub con CI (pytest en cada push). Resultado: cada capa se modifica y escala por separado.

🔪 **Para explayarte:** el ejemplo concreto que siempre funciona: *"Si mañana llega un archivo con 500 registros corruptos, el enfoque tradicional los mete a la base o los pierde en silencio. Nuestro pipeline los rechaza, los guarda con el motivo, y el embudo del dashboard te muestra cuántos fueron. Eso es DataOps: el error es un dato más, no una sorpresa."* - ¿Por qué idempotente? → puedo correr el pipeline 10 veces y la base queda igual: reproducibilidad, principio DataOps. - ¿Por qué 0 nulos? → la limpieza imputó/normalizó y la validación verificó; el modelo nunca ve basura.

Lámina 03 — El reto de negocio (26,5% churn)

📌 **Ancla:** *"El error caro no es la falsa alarma: es el cliente que se fue sin que nadie lo llamara."*

💡 **El concepto.** Churn = tasa de abandono. Es un problema de **clasificación binaria supervisada**: tenemos la respuesta histórica (quién se fue) y queremos anticipar la futura. El insight clave del negocio: **retener cuesta menos que captar** (un incentivo vs marketing + comisión + onboarding). Y el insight técnico que define TODO el proyecto: la clase está **desbalanceada** (26,5% churn / 73,5% no churn), y eso determina qué métrica importa y cómo entrenar.

📖 **La historia.** 7.043 clientes: 1.869 abandonan, 5.174 permanecen. Cuando hicimos el análisis bivariado, los factores de riesgo saltaron a la vista: contrato **mes a mes** (42,7% churn vs 2,8% en contrato de 2 años), **fibra óptica** (41,9% — probablemente por precio/competencia), **cheque electrónico** como pago (45,3%), y sobre todo **antigüedad baja** (0-12 meses: 47,4% — casi la mitad del primer año se va). Estos números no van en el deck recortado, pero los dices tú — y reaparecen solos en la demo del cliente nuevo, cuando el modelo explica "por qué".

🔪 **Para explayarte:** la asimetría de costos es tu argumento estrella: *"Un falso positivo cuesta un descuento ofrecido a alguien que no se iba. Un falso negativo cuesta el cliente completo — su facturación anual entera. Por eso optimizamos recall y no accuracy."* Si preguntan por qué supervisado: *"porque el dataset ya trae la etiqueta; no necesitamos descubrir grupos (no supervisado), necesitamos predecir una respuesta conocida."*

Lámina 04 — Entrenamiento y comparación (recall por modelo)

📌 **Ancla:** *"Cuatro modelos compitieron con las mismas reglas; ganó el que detecta más fugas y además se puede explicar."*

💡 **El concepto.** Metodología de comparación honesta: **mismos datos, misma partición, mismas métricas**. Entrenamos 2 modelos base (lo que enseña el material: Regresión Logística y Árbol de decisión con parámetros por defecto) y 2 mejoras (Random Forest y LogReg **balanceada**). El desbalance se maneja con `class_weight='balanced'`: en vez de inventar datos sintéticos (SMOTE), le dice al modelo "equivocarte con la

clase minoritaria cuesta $\sim 3\times$ más" (pondera inverso a la frecuencia). SMOTE lo **evaluamos y descartamos**: agrega dependencia (imbalanced-learn), riesgo de distorsión, y el class_weight logró el mismo objetivo — esa es una "alternativa evaluada" que les encanta preguntar.

📖 **La historia.** Resultados sobre el test (2.113 clientes): LogReg balanceada **79,7%** de recall (ganadora), Random Forest 63,5%, Árbol 60,6%, LogReg base 56,0%. Lo que hay que saber contar: la LogReg base y la balanceada son **el mismo algoritmo** — la única diferencia es el manejo del desbalance, y el recall saltó de 56% a 79,7%. Esa es la evidencia de que **el desbalance era el problema**, no el algoritmo. Elegimos por **F1 + recall** (no por accuracy): la balanceada tiene el mejor F1 (0,62) y el mayor recall. Bonus decisivo: es **interpretable** — sus coeficientes nos dicen qué variable empuja y cuánto, cosa que usamos después en [/predict/nuevo](#) para explicar cada predicción.

Lo que dices verbal (estaba en la lámina eliminada): partición **70/30 estratificada** — train 4.930 / test 2.113, y [stratify](#) mantiene el 26,5% de churn en ambos conjuntos (sin eso, la evaluación sería una lotería). El preprocesamiento (StandardScaler para numéricas + OneHotEncoder para categóricas) vive **dentro** del Pipeline de sklearn: se ajusta solo con train (evita fuga de datos) y viaja serializado con el modelo (producción transforma idéntico al entrenamiento).

🔪 **Para explayarte:** si preguntan por qué no accuracy: "con 73,5% de mayoría, un modelo que dice 'nadie se va' saca 73,5% de accuracy y cero valor. La accuracy premia la pereza en clases desbalanceadas." Si preguntan por qué no un modelo más complejo (XGBoost, redes): "con 7.043 filas y necesidad de explicabilidad ante el negocio y la ley, un modelo lineal bien tratado rinde igual o mejor y se defiende solo."

Lámina 05 — Métricas e interpretación (matriz de confusión)

📌 **Ancla:** "De los 561 que se van, detectamos 447 y se nos escapan 114 — y sabemos exactamente cuánto cuesta cada celda."

🟡 **El concepto.** La matriz de confusión es el mapa de los 4 destinos posibles de una predicción binaria. Con nuestros números de test (2.113): - **TN = 1.121**: dijimos "se queda" y se quedó ✓ - **FP = 431**: dijimos "se va" y se quedó — falsa alarma, cuesta un incentivo - **FN = 114**: dijimos "se queda" y SE FUE — el error caro, cliente perdido - **TP = 447**: dijimos "se va" y se iba — fuga detectada a tiempo ✓

De ahí salen todas las métricas: **Recall** = $TP/(TP+FN) = 447/561 = 79,7\%$ (de los que se van, cuántos capturo). **Precision** = $TP/(TP+FP) = 447/878 = 50,9\%$ (de mis alarmas, cuántas eran reales — 1 de cada 2). **F1 = 0,62** (media armónica: castiga si una de las dos se derrumba). **Accuracy = 74,2%** — y aquí la trampa: predecir "nadie se va" da 73,5%. Nuestro modelo "solo" le gana por 0,7 puntos en accuracy... pero detecta 447 fugas contra **cero**. La accuracy no distingue valor.

Gini y ROC: la curva ROC muestra el trade-off detección vs falsas alarmas en **todos los umbrales posibles**. AUC = área bajo esa curva = 0,846 = "si tomo un cliente que se va y uno que se queda al azar, el 84,6% de las veces el modelo le da más probabilidad al que se va". **Gini = 2·AUC - 1 = 0,693** (misma información, escala 0 = azar, 1 = perfecto). Gini $\approx 0,7$ = poder de ordenamiento sólido → el negocio puede **rankear** la cartera y llamar primero al top de riesgo.

📖 **La historia.** Estas métricas salen del **holdout** — el 30% que el modelo evaluador nunca vio. Y un detalle de honestidad técnica que nos gusta contar: el modelo **final** que quedó en producción se re-entrena con el 100%

de los datos (más dato = mejor modelo final), pero las métricas que reportamos son las del holdout — nunca infladas. El dashboard replica esto: KPIs desde `modelo_artefacto` (holdout), scoring sobre la base completa, cada uno etiquetado.

🔑 **Para explayarte:** si preguntan "¿50,9% de precision no es malo?": *"Es el trade-off elegido: en retención la falsa alarma cuesta un descuento; el abandono no detectado cuesta el cliente. Y no contactamos a ciegas: ordenamos por probabilidad y el negocio decide hasta dónde llegar con su presupuesto — para eso sirve el Gini."* Si piden subir precision: mover el umbral de decisión (está en las mejoras).

Lámina 06 — Un hallazgo clave (overfitting)

📌 **Ancla:** *"El Random Forest se aprendió el entrenamiento de memoria: 100% en train, 63,5% en test. La LogReg rinde igual en ambos: eso es generalizar."*

💡 **El concepto. Overfitting** = el modelo memoriza el ruido del entrenamiento en vez de aprender el patrón. Se diagnostica con UNA comparación: rendimiento en train vs test. Brecha grande = memorización. El Random Forest sin regular (árboles profundos, sin límite) tiene capacidad de sobra para memorizar 4.930 filas → recall 100% en train que se derrumba a 63,5% en test: **-36,5 puntos** de brecha. La Regresión Logística, con su frontera lineal (poca capacidad de memorizar), va de 80,4% en train a 79,7% en test: brecha de **0,7 puntos** — generalización de libro.

📖 **La historia.** Esto NO fue un accidente que ocultamos — lo **convertimos en hallazgo** (indicador #6: limitaciones con evidencia y reflexión crítica). La reflexión: más capacidad no es mejor modelo; con datasets chicos, los modelos flexibles necesitan **regularización** (limitar `max_depth`, `min_samples_leaf`) y eso quedó en el plan de mejoras. También explica por qué el "mejor" modelo en papers (ensembles) no ganó aquí: el contexto (tamaño del dato, interpretabilidad requerida) manda.

🔑 **Para explayarte:** la analogía que siempre aterriza: *"Es el alumno que memoriza el ensayo y saca 7, pero en la prueba con preguntas nuevas saca 4. El que entendió la materia saca 6 en ambas. Nosotros elegimos al que entendió."* Si preguntan cómo lo prevenimos en general: holdout estratificado siempre, y validación cruzada k-fold como mejora (da la métrica con intervalo de confianza).

Lámina 07 — Seguridad y gobernanza (Ley 21.719)

📌 **Ancla:** *"No le pusimos seguridad al sistema al final: el sistema nació con ella — compliance by design."*

💡 **El concepto.** Dos capas distintas que se preguntan por separado: - **Seguridad** = proteger el dato: los 4 frentes de nuestra auditoría son **(1) Credenciales** (0 secretos en código ni en historial git — verificado escaneando; todo vive en variables de entorno, `.env` en gitignore), **(2) Accesos** (RLS = Row-Level Security en TODAS las tablas, cerradas por defecto: los roles públicos `anon` / `authenticated` no leen nada; y un rol `telco_lectura` de solo lectura para BI = **privilegio mínimo**), **(3) Entorno** (escaneo `pip-audit`: 10 CVEs detectados en dependencias, con plan de actualización; `trivy` propuesto para imágenes), **(4) Logs** (accesos fallidos repetidos = posible fuerza bruta). - **Gobernanza** = gestionar el dato como activo: **linaje** (raw → clean → validated → BD, cada zona con timestamp: puedo reconstruir el camino de cualquier registro), **políticas de acceso por rol** (dueño escribe / lectura consume / anon bloqueado), **versionado del modelo**

(`modelo_artefacto` guarda modelo + métricas + fecha: sé qué modelo estaba en producción y cuándo), **derechos del titular** (todo indexado por `customer_id` → ARCO ejecutable), **matriz de riesgos** (credenciales, CVEs, free tier, deriva).

La ley: la **21.719** moderniza la 19.628, vigencia plena **01-12-2026**, estándar tipo GDPR con Agencia de Protección de Datos y multas reales. Nuestro dataset tiene **datos personales** (género, edad, situación familiar, datos económicos) pero **no sensibles** en sentido estricto (no hay salud, biometría, religión...). Principios que cumplimos por diseño: **finalidad** (los datos solo predicen churn), **minimización** (customerID es un código, no nombre/RUT; solo variables necesarias), **seguridad** (todo lo anterior), **responsabilidad proactiva** (la auditoría documentada ES la evidencia).

📖 **La historia.** El escaneo de secretos no fue teatro: cada commit del proyecto pasó por revisión antes de subir. Y los 10 CVEs los encontramos nosotros con pip-audit (starlette, python-dotenv, pytest) — mostrarlos con plan de actualización vale más que decir "no hay vulnerabilidades" (nadie te lo cree).

🔪 **Para explayarte:** pregunta trampa habitual: "¿cifran los datos?" → "En tránsito sí: TLS en toda la cadena (API, BD, dashboard). En reposo, Supabase cifra el storage subyacente. Y el dato más sensible que NO ciframos... no existe: minimizamos — no guardamos nombre ni RUT." Pregunta ARCO: "un cliente pide borrar sus datos" → "DELETE por customer_id en clientes y predicciones; el diseño indexado lo hace trivial. Y el modelo no memoriza individuos: es un modelo lineal de patrones agregados."

Lámina 08 — Integración con BI + DEMO

📌 **Ancla:** "El dashboard no es una foto: es la ventana en vivo a la base de datos — y el modelo corre detrás como microservicios."

💡 **El concepto.** La arquitectura de serving separa **entrenar** de **predecir** (patrón train/inference): son cargas distintas (una pesada y ocasional, otra liviana y constante) que no deben compartir destino. Nuestros dos microservicios usan **la misma imagen Docker** y se diferencian por una variable de entorno `ROL` — el mismo patrón "un contenedor, distinta responsabilidad" del pipeline, extendido a la capa de IA. Se comunican **solo vía Supabase** (tabla `modelo_artefacto`): el trainer entrena y guarda el modelo serializado; el predictor lo carga y predice **sin re-entrenar jamás**. Cero acoplamiento: puedo tumbar el trainer y el predictor sigue sirviendo.

📖 **La historia y el flujo de la demo** (guion mental, 3 min): 1. "**Vaciar TODO**" → el dashboard queda en blanco (KPIs "—"): demuestra que no hay nada precargado ni pantallazos. 2. **Entrenar** (botón o Swagger del trainer, `POST /train`) → el modelo se entrena en la nube y se guarda en Supabase; el dashboard **se llena solo** (auto-refresh cada 3 s detecta el artefacto) → aparecen los KPIs del holdout. 3. **Predecir** (`POST /predict/batch`) → 7.043 puntuados, 2.914 en riesgo → tabla `predicciones` → gráficos completos. 4. **Cliente nuevo** (`POST /predict/nuevo` desde el formulario): preset "alto riesgo" → **84%**; preset "fiel" → **1,4%**. El modelo **nunca vio** a estos clientes — y explica el porqué (factores con peso: "antigüedad 2 meses +1,65", "contrato dos años -0,81"). Eso demuestra **generalización + interpretabilidad** en una sola pantalla.

Las 4 vistas del panel y para qué existe cada una: **KPIs** (salud del modelo), **matriz de confusión** (dónde se equivoca), **churn por segmento** (dónde está el riesgo de negocio), **tabla de errores caso a caso** (análisis fino: puedo filtrar los falsos negativos y ver qué tienen en común), **embudo del pipeline** (el dato también se monitorea).

🔧 **Para explayarte:** por qué Streamlit y no Power BI: "versionable en git, desplegable como app web con URL, y programable — el formulario del cliente nuevo no existe en un BI de arrastrar y soltar." Por qué las métricas del panel son creíbles: "KPIs desde el holdout guardado con el modelo; scoring sobre la base completa; cada número dice su origen."

Lámina 09 — Infraestructura requerida

📌 **Ancla:** "No adivinamos los recursos: los medimos. 200 MB y 10% de CPU reales → pedimos 2-4 GB y sobra margen 10×."

💡 **El concepto.** Un requerimiento de infraestructura serio tiene 5 dimensiones: **cómputo, almacenamiento, red, disponibilidad y redundancia** — y se dimensiona desde mediciones, no desde el miedo. La decisión **nube vs on-premise** se argumenta por: costo inicial (cero vs comprar servidores), elasticidad (nuestro entrenamiento dura ~1,3 s — hardware dedicado sería un desperdicio), y servicios gestionados (Postgres con backups y TLS sin operarlo nosotros). On-premise solo se justificaría por soberanía de datos estricta — y la Ley 21.719 **no** exige territorialidad si el tratamiento cumple garantías.

📖 **La historia.** Cada número de la lámina tiene una medición detrás (benchmark con `psutil` + `time`): el sistema completo procesó los 7.043 registros con ~200 MB de RAM, 10% CPU, 5,3 s total. De ahí: 1-2 vCPU / 2-4 GB por servicio = margen 10×. El hallazgo del benchmark que moldeó el diseño: **la red domina** — leer de la nube fue ~219× más lento que local (3,94 s vs 0,018 s), el cómputo era lo de menos. Consecuencia directa en el requerimiento: **servicios y BD en la misma región** (colocalizar). La **redundancia** se piensa desde los puntos de fallo: el predictor es **stateless** (el modelo vive en la BD) → replicarlo es trivial, ≥2 réplicas tras balanceador → 99,9%. El único punto único de fallo real es la BD → réplica de lectura + backups verificados.

La historia de guerra que hace única tu respuesta de dependencias: "fijamos versiones porque lo vivimos: un rebuild sin `numpy` fijado resolvió `numpy 2.x`, incompatible binario con `pandas 2.1.4`, y el dashboard se cayó con `Segmentation fault` en producción. Lo diagnosticamos por logs, fijamos `numpy 1.26.4` y documentamos el pin. `scikit-learn` también va fijado (1.9): el modelo serializado exige la misma versión al deserializar." Eso es un requerimiento de software aprendido con sangre, no copiado de un manual.

🔧 **Para explayarte:** ¿escala a 1M de clientes? → "El diseño sí: predictor **stateless** se replica horizontal; el pipeline procesa por lotes y se particiona; a esa escala sumaría un orquestador (Airflow) manteniendo las mismas etapas. Lo que cambiaría primero es la BD: particionar `predicciones` por fecha."

Lámina 10 — Estrategia de monitoreo

📌 **Ancla:** "Tres preguntas: ¿está viva la solución? ¿procesa bien el dato? ¿sigue prediciendo bien? — y la tercera es la que falla en silencio."

💡 **El concepto.** Monitorear un sistema de IA es más que monitorear un servidor, por eso **3 planos**: 1. **Infraestructura** — disponibilidad y salud técnica (latencia, errores, RAM/CPU). 2. **Datos** — calidad del flujo (% rechazados, duración de corridas). Un cambio brusco aquí anticipa problemas del modelo. 3. **Modelo** — el plano que casi todos olvidan: **deriva (drift)**. El modelo se entrenó con datos de una época; si el mundo cambia (nueva promoción cambia el mix de contratos, cambia el perfil de clientes), los datos de producción dejan de parecerse

a los de entrenamiento y el rendimiento cae **sin que nada "falle"**: la API responde 200, el dashboard se ve lindo, y las predicciones valen cada vez menos. Se detecta con **PSI** (Population Stability Index: compara la distribución de cada variable hoy vs entrenamiento) y con **recall re-etiquetado** (esperar el churn real del mes y medir cuántos anticipamos).

📖 **La historia — qué existe HOY vs qué proponemos** (decir esta distinción con orgullo, es honestidad técnica): - **Hoy (implementado y demostrable)**: `GET /health` en cada microservicio — Railway lo usa como health check y **reinicia automáticamente** ante fallo (`restartPolicyType: ON_FAILURE` — así se recuperó solo el dashboard cuando el segfault); `carga_logs` como auditoría por corrida; benchmark psutil documentado; el dashboard auto-refresca cada 3 s reflejando el estado real de la BD; **CI en GitHub Actions** corre pytest en cada push (ese es el rol que el enunciado le da a Jenkins — mismo concepto, herramienta distinta). - **Propuesto (para operación 24/7 organizacional)**: Prometheus + Grafana con exporters por servicio; alertas proactivas a Slack/correo (p95 > 2 s, error rate > 1%, rechazados > 5%); y el job mensual de deriva: si recall < 70% → disparar re-entrenamiento.

🔪 **Para explayarte**: ¿por qué no instalaron Prometheus ya? → "Porque el valor del monitoreo continuo aparece con operación 24/7 y usuarios reales; en un proyecto académico habría sido decorado. Preferimos implementar el monitoreo que **SÍ** se usa hoy (health checks, logs de auditoría, CI) y diseñar el resto con umbrales concretos." Esa respuesta convierte una ausencia en una decisión.

Lámina 11 — Despliegue organizacional (4 fases)

📌 **Ancla**: "No se lanza un modelo a toda la empresa un lunes: se gana la confianza por fases, con criterio de avance medible en cada una."

🔴 **El concepto**. La adopción progresiva minimiza el impacto operacional y gestiona el riesgo del cambio. Cada fase tiene: alcance acotado, usuarios definidos y un **criterio de avance** (si no se cumple, no se avanza — eso la hace un plan y no una lista de deseos): 1. **Piloto (4-6 semanas)**: el equipo de retención usa el dashboard (solo lectura) y contacta manualmente el top-100 de riesgo. Criterio: la retención del grupo contactado supera al **grupo control** (clientes de riesgo similar NO contactados — sin control no puedes atribuir el resultado al modelo). 2. **Integración CRM**: la ficha del cliente consulta `POST /predict/nuevo` — el ejecutivo en llamada ve riesgo y factores en vivo. Criterio: adopción de los ejecutivos + latencia < 1 s. 3. **Automatización**: scoring batch semanal alimenta campañas segmentadas. Criterio: ROI de campaña > costo de incentivos. 4. **Operación continua**: re-entrenamiento programado + monitoreo de deriva + comité de gobernanza.

📖 **La historia — por qué nuestro diseño hace esto fácil**: la **API REST es el contrato**: el CRM no necesita saber de Python ni sklearn, hace un POST y recibe JSON — el endpoint ya existe y lo demostramos en vivo. La **migración de datos** toca solo la etapa de ingesta (de CSV a la fuente transaccional); limpieza, validación y carga no se tocan — esa es la recompensa del diseño por etapas. El **rollback** ya está resuelto: `modelo_artefacto` versiona modelo + métricas + fecha; si una versión nueva degrada, el predictor recarga la anterior con `/reload` — sin redespigüe. **Capacitación por rol**: ejecutivos (leer el riesgo y el porqué), analistas (interpretar métricas), TI (operación y alertas).

🔪 **Para explayarte**: el cierre de negocio: "Con recall 79,7%, de cada 100 fugas anticipamos ~80. Si la telco pierde 1.869 clientes al año y retiene aunque sea el 10% de los detectados, son ~150 clientes × su facturación

anual — contra una infraestructura que cuesta unos pocos dólares al mes. El ROI no es la duda; la duda es cuánto tardas en confiar en el modelo, y para eso es el piloto con grupo control."

Lámina 12 — Cierre (limitaciones y mejoras)

 **Ancla:** "Sistema completo, honesto y mejorable — y cada mejora tiene su plan, no es una lista de buenas intenciones."

 **El concepto.** Limitación $\neq$ error: es un límite conocido, **con evidencia**, del que deriva una mejora concreta. La pareja limitación→mejora→cómo implementarla es exactamente lo que piden los indicadores #6 y #10.

 **Las 4 parejas que hay que dominar:** 1. **Precisión 50,9%** (1 de 2 alarmas es falsa) → **ajustar el umbral de decisión:** hoy el corte es 0,5; barriendo el umbral sobre el holdout puedo elegir el punto que optimice F1 o el costo de negocio. Un día de trabajo, sin re-entrenar. *Primera por impacto/costo.* 2. **RF sobreajusta** (100→63,5) → **regularizar:** búsqueda de hiperparámetros (`max_depth`, `min_samples_leaf`) con validación cruzada. 3. **Dataset chico y desbalanceado** (7.043; 26,5%) → **validación cruzada k-fold** (k=5, en el modo `--eval`): la métrica deja de ser un número y pasa a ser un intervalo — sabes cuánto confiar en ella. Más **feature engineering** (ej: cargos/antigüedad, cambios de plan). 4. **10 CVEs + free tier** → actualizar dependencias (python-dotenv ya, FastAPI coordinado con re-test) y plan pago/keep-alive en producción.

El golpe final que nadie espera: "varias mejoras propuestas en parciales anteriores ya las implementamos: separar entrenamiento de inferencia (microservicios trainer/predictor — era mejora en Ev3, hoy es arquitectura desplegada), el rol de solo lectura, y la predicción de clientes nunca vistos con factores explicativos." Eso demuestra que las mejoras no son retórica: el proyecto tiene historial de proponerlas y cumplirlas.

 **Para explayarte (cierre de la presentación):** "Entrenamos un modelo honesto — 79,7% de recall medido donde corresponde—, lo auditamos contra la ley que entra en vigencia este año, lo medimos antes de dimensionarlo, y lo dejamos operando con su plan de monitoreo y despliegue. El sistema completo está en vivo y lo pueden visitar ahora mismo."

Apéndice — PMBOK (no tiene lámina, pero cae en preguntas)

 **Ancla:** "PMBOK híbrido: hitos fijos por fuera (las evaluaciones), sprints adaptativos por dentro (el feedback del docente)."

Los 5 grupos de procesos aplicados de verdad: **Inicio** (caso de negocio del churn, selección del caso Telco), **Planificación** (EDT por fases: pipeline → modelo → operación; matriz de riesgos), **Ejecución** (sprints semanales; commits descriptivos como evidencia), **Monitoreo y control** (CI con pytest; la revisión del docente por parcial fue nuestro control de calidad externo), **Cierre** (el EFT: informe consolidado + defensa + lecciones aprendidas). **Ejemplo concreto de lo adaptativo:** el hito "Parcial 3" fijaba el entregable, pero el cómo iteró — baseline → balanceado → microservicios. **Riesgos gestionados reales:** free tier que se duerme (verificación pre-demo), sobreajuste (holdout), fuga de credenciales (.env + escaneo). Herramienta: GitHub (historial = sprints, releases = hitos); en una organización: Jira.

Cómo estudiar con este material

1. **Primera pasada:** lee todo de corrido (30 min) — la historia tiene un arco: dato → modelo → operación.
2. **Segunda pasada:** tapa el contenido y, con solo el título de cada lámina, recita el  ancla y 3 ideas. Lo que no salga, reléelo.
3. **Tercera pasada (con Diego):** uno dice "lámina 7" y el otro habla 60 segundos sin parar. Luego intercambian.
4. La noche antes: solo las  anclas y los números gordos: **7.043 · 26,5% · 70/30 · 79,7% · 0,62 · 0,693/0,846 · 447/114/431/1.121 · 2.914 · 100→63,5 · ~219× · 200 MB/10% · 10 CVEs · 01-12-2026.**